

Linked Lists and Iterators

4/9/25

Green slides taken (w/ minor modifications) from Cynthia Lee's CS 2 slides on <http://www.peerinstruction4cs.org>, licensed under [Creative Commons Attribution-NonCommercial 4.0 International License](#).

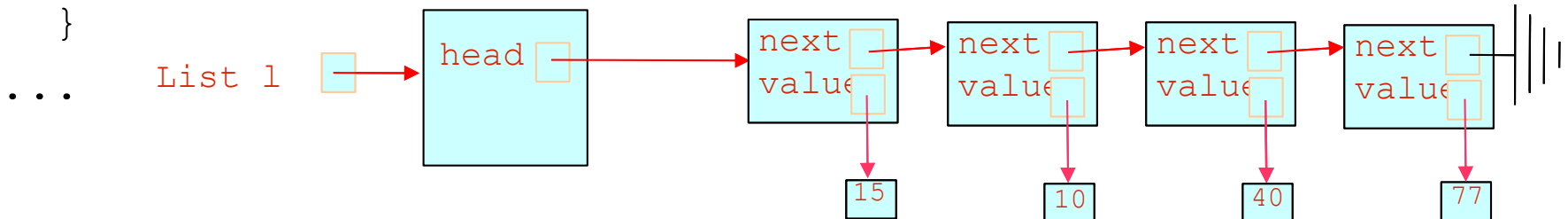
Recall: LinkedList

```
public class OurLinkedList<T> implements List<T> {  
    private Node head;  
    //optionally: could have size
```

```
    public class Node {  
        public T value;  
        public Node next;
```

An “inner class”.
Objects can only be
created in context of
the main class.

```
        public Node(T value, Node next) {  
            this.value = value;  
            this.next = next;  
        }  
    }  
}
```



Which of the following advances curr to point to the next Node? (i.e. what should go in the blank?)

```
public int size() {  
    Node curr = head;  
    int count = 0;  
    while(curr != null) {  
        count++;  
        _____  
    }  
    return count;  
}
```

A. curr++;

B. curr.next;

C. curr = curr.next;

D. Node temp = curr.next;
 curr = temp;

E. Not exactly one of the above

Which of the following advances curr to point to the next Node? (i.e. what should go in the blank?)

```
public int size() {  
    Node curr = head;  
    int count = 0;  
    while(curr != null) {  
        _____  
    }  
    return count;  
}
```

A. curr++;

B. curr.next;

C. curr = curr.next;

D. Node temp = curr.next;
 curr = temp;

E. Not exactly one of the above
(C & D)

Suppose you want to print the contents of a `OurLinkedList` for debugging. You want a variable `curr` to refer to each `Node` in turn. Which of the following does this correctly?

A. `Node curr = head;`
`while(curr != null) {`
 ...
 `curr = curr.next;`
`}`

B. `Node curr = head;`
`while(curr.next != null) {`
 ...
 `curr = curr.next;`
`}`

C. `Node curr = head.next;`
`while(curr != null) {`
 ...
 `curr = curr.next;`
`}`

D. `Node curr = head.next;`
`while(curr.next != null) {`
 ...
 `curr = curr.next;`
`}`

E. Not exactly one of the above

Suppose you want to print the contents of a `OurLinkedList` for debugging. You want a variable `curr` to refer to each `Node` in turn. Which of the following does this correctly?

A. `Node curr = head;`
`while(curr != null) {`
`...`
`curr = curr.next;`
`}`

B. `Node curr = head;`
`while(curr.next != null) {`
`...`
`curr = curr.next;`
`}`

C. `Node curr = head.next;`
`while(curr != null) {`
`...`
`curr = curr.next;`
`}`

D. `Node curr = head.next;`
`while(curr.next != null) {`
`...`
`curr = curr.next;`
`}`

E. Not exactly one of the above

```
public boolean contains(T s) {  
    //returns whether the list contains value s
```

Traversing a list

Array-based

```
for(int i=0; i < num; i++) {  
    operation on vals[i]  
}
```

Linked list

```
Node curr = head;  
while(curr != null) {  
    operation on curr.val  
    curr = curr.next;  
}
```


Iterator ADT

- boolean hasNext()
- T next() //returns and advances
- void remove() //optional: removes most recent
- ListIterator: hasPrevious, previous, nextIndex, previousIndex, add(T), set(T) (optional)

Common pattern with an iterator

```
List<T> stuff = ...; // or other Collection
```

```
Iterator<T> it = stuff.iterator();
```

```
while(it.hasNext()) {
```

```
    T val = it.next();
```

```
    //do something with val
```

```
}
```

Common pattern(s) with an iterator

```
List<T> stuff = ...; // or other Collection
```

```
Iterator<T> it = stuff.iterator();
```

```
while(it.hasNext()) {
```

```
    T val = it.next();
```

```
    //do something with val
```

```
}
```

```
for(T val : stuff) {
```

```
    //do something with val
```

```
}
```

How many of the following work as the body for the clear method (empties list)?

I. `size = 0;`

A. 0

II. `head = null;`

B. 1

III. `head.next = null;`

C. 2

IV. `Node curr = head;`

D. 3

`while(curr != null) {`

E. 4

`curr.value = null;`

`curr = curr.next;`

`}`

How many of the following work as the body for the clear method (empties list)?

I. size = 0;

A. 0

II. head = null;

B. 1 (II.)

III. head.next = null;

C. 2

IV. Node curr = head;

D. 3

while(curr != null) {

E. 4

curr.value = null;

curr = curr.next;

}

What is wrong with this add method?

```
public void add(T toAdd) {    //add to end of the list
    Node curr = head;
    while(curr.next != null)
        curr = curr.next;
    Node newNode = new Node(toAdd, null);
    curr.next= newNode;
}
```

- A. Syntax error
- B. Doesn't add the new value
- C. Throws exception if list is empty
- D. Removes existing value(s) from the list
- E. Not exactly one of the above

What is wrong with this add method?

```
public void add(T toAdd) {    //add to end of the list
    Node curr = head;
    while(curr.next != null)
        curr = curr.next;
    Node newNode = new Node(toAdd, null);
    curr.next= newNode;
}
```

- A. Syntax error
- B. Doesn't add the new value
- C. Throws exception if list is empty
- D. Removes existing value(s) from the list
- E. Not exactly one of the above

Write get method

(returns value at given index or throws `IndexOutOfBoundsException`)

```
public T get(int index) {
```

```
    ...
```